MedIQ BI: An AI-Powered Hospital Analytics Dashboard

A comprehensive Business Intelligence mini-project

Anand More
Department of Computer Engineering
Pimpri Chinchwad College of Engineering and Research , Ravet
moreanand111011@gmail.com

March 29, 2026

Abstract

This report details the development and implementation of MedIQ BI, an end-to-end hospital analytics dashboard designed to visualize and manage healthcare data efficiently. The system comprises a Node.js backend for data processing and a React-based frontend utilizing a custom Vanilla CSS design system. Key features include tracking patient inflow, monitoring revenue streams, analyzing disease distributions, evaluating doctor performance, and providing AI-driven resource allocation insights. The application is fueled by a simulated dataset of 1,000 patient records, demonstrating its capability to handle structured healthcare data in real-time.

Contents

Chapter 1

Introduction

Modern healthcare facilities generate vast amounts of data daily, ranging from patient admissions and doctor schedules to billing and disease tracking. Navigating this data efficiently requires robust Business Intelligence (BI) tools. The MedIQ BI project aims to bridge the gap between raw hospital data and actionable insights by providing an intuitive, premium, and responsive web-based dashboard.

The primary objectives of the system are to:

- Visualize historical patient inflow to identify peak operational hours.
- Provide a granular overview of doctor performance and patient satisfaction.
- Analyze financial trends and revenue distribution.
- Utilize predictive AI deployment insights to suggest resource allocation automatically.

Chapter 2

System Architecture

The system follows a standard client-server architecture, decoupled to allow independent scaling and development.

2.1 Backend Services

The backend is built using **Node.js** and **Express.js**. It serves as the data processing pipeline. A custom data generator script creates a synthetic dataset (`hospital_data.csv`) containing precisely 1,000 records of patient visits, mapping attributes such as timestamp, assigned doctor, department, diagnosis, revenue, and wait times. The RESTful API parses this CSV file in memory and aggregates the data to serve summarized Key Performance Indicators (KPIs) and chart-ready structures to the frontend.

2.2 Frontend Application

The client side is developed using **React (Vite)** and employs a modern, Vanilla CSS-based design system. The architecture relies on component-based development:

- **Recharts**: Utilized for generating SVG-based data visualizations, customized with CSS properties for premium rendering.
- **Framer Motion**: Integrated to provide smooth, micro-animations and staggered entrance effects, enhancing user experience.
- **Lucide Icons**: Used for clear, scalable vector taxonomy across the navigation and KPI cards.

Chapter 3

Implementation Details

3.1 Data Generation and Processing

A deterministic algorithm generates the 1,000-sample dataset, ensuring realistic bounds on random values (e.g., revenue caps, logical department-to-diagnosis mappings). The Node.js server aggregates this data to compute:

- Total patient count and average waiting times.
- Revenue summations over a 6-month period.
- Grouping frequencies for Disease Classification and Doctor allocation.

3.2 User Interface and Design System

Moving away from utility-class dependencies, a custom Vanilla CSS framework was implemented across the application. It relies heavily on:

- **CSS Custom Properties**: Variables representing `--primary`, `--surface-color`, and `--shadow-md` enable dynamic theming and consistent aesthetics.
- **Glassmorphism**: Advanced `backdrop-filter` layers create a modern sense of depth across the sidebar and metric cards.

Chapter 4

Results and Visualizations

The resulting application successfully retrieves and plots the 1,000-record dataset into several core components.

4.1 Patient Inflow Chart

An Area chart displaying dynamic admission frequencies. Custom SVGs and localized gradients provide immediate visual feedback on institutional loads over a 6-month timeline.

4.2 Top Performing Doctors Table

A tabular component integrated with inline progression bars, allowing administrators to instantly scan capability gaps, patient counts, and doctor availability.

4.3 Disease Distribution Donut

A customized pie chart illustrating the categorization of ailments, color-mapped for high contrast and readability.

4.4 AI Deployment Insight

A highlighted card presenting actionable, algorithmically suggested directives, such as deploying additional standby nurses to specific wards during predicted emergency surges.

Chapter 5

Conclusion

The MedIQ BI mini-project effectively demonstrates the synthesis of modern web technologies to create functional, premium operational dashboards. By tightly coupling a Node.js data pipeline with a performant, animated React interface, the project provides a scalable foundation for extensive hospital administration and business intelligence tracking. Future enhancements may include live database integration, real-time WebSocket communication for live updates, and integration with actual predictive Machine Learning models.